

Teoría de la información y codificación

Trabajo práctico de simulación: Codificación de fuente y canal

Fecha de Entrega: 17/06/2026



**FACULTAD
DE INGENIERÍA**



**UNIVERSIDAD
NACIONAL
DE LA PLATA**

Alumno: Ramirez Tolentino, Fernando (01964/8)

Índice

1. Introducción.....	3
2. Ejercicio 1: Código de bloque lineal (14,10) como corrector.....	3
2.1. Inciso i: Diseño analítico y parámetros teóricos.....	3
2.1.1. Capacidad de corrección de errores (t_c).....	3
2.1.2. Diseño de la matriz de verificación de paridad (HT).....	4
2.1.3. Diseño de la matriz generadora (G).....	5
2.1.4. Capacidad de detección de errores (t_d).....	6
2.1.5. Ganancia asintótica de codificación (G_a).....	6
2.2. Inciso ii: Simulación y relevamiento de curvas.....	7
2.3. Inciso iii: Análisis de desempeño.....	9
3. Ejercicio 2: Código de bloque lineal (14,10) como detector.....	10
3.1. Inciso i: Lógica de detección y parámetros operativos.....	10
3.2. Inciso ii: Simulación y relevamiento de curvas.....	11
3.3. Inciso iii: Análisis de desempeño.....	12
4. Ejercicio 3: Codificación de fuente mediante algoritmo de Huffman.....	13
4.1. Implementación del algoritmo y estimación de probabilidades.....	13
4.2. Inciso i: Longitud promedio y tasa de compresión.....	13
4.3. Inciso ii: Análisis de compresión con fuente extendida.....	14
5. Conclusiones.....	15
Apéndice A. Implementación y equivalencia de lenguajes.....	15
Apéndice B. Guía de ejecución de los simuladores.....	17
B.1. Instalación de dependencias.....	17
B.2. Estructura de directorio.....	17
B.3. Ejecución y resultados esperados.....	17

1. Introducción

El presente informe tiene como objetivo la evaluación y validación de técnicas fundamentales de codificación de fuente y de canal en un sistema de comunicación digital. Para ello, se desarrollaron entornos de simulación computacional que permiten contrastar los modelos teóricos de probabilidad de error y compresión de datos frente a escenarios reales modelados algorítmicamente.

En la primera etapa, se analiza el desempeño de un código de bloque lineal (14,10) operando sobre un canal con Ruido Blanco Gaussiano Aditivo (AWGN) y modulación BPSK. Se relevaron las curvas de probabilidad de error operando el código bajo dos paradigmas distintos: como esquema corrector de errores y como detector estricto para descarte de paquetes.

En la segunda etapa, se aborda la codificación de fuente mediante la implementación del algoritmo de compresión de Huffman. El análisis se centra en la explotación de la redundancia espacial de una imagen digital, demostrando cómo la extensión de la fuente permite alcanzar tasas de compresión significativas incluso cuando la distribución marginal de los símbolos base aparenta ser de máxima entropía.

Las consideraciones sobre el lenguaje informático utilizado, la justificación de las librerías matriciales y la equivalencia exacta de variables con el modelo sugerido por la cátedra se detallan en el **Apéndice A**. Asimismo, para facilitar la revisión y reproducción de los resultados por parte de los evaluadores, se incluye una guía de configuración y ejecución de los simuladores en el **Apéndice B**.

2. Ejercicio 1: Código de bloque lineal (14,10) como corrector

2.1. Inciso i: Diseño analítico y parámetros teóricos

2.1.1. Capacidad de corrección de errores (t_c)

El objetivo es diseñar un código de bloque lineal $(n, k) = (14, 10)$ sistemático. Antes de proponer las matrices generadora y de control de paridad, se evalúa la capacidad máxima de corrección de errores (t_c) del código mediante la Cota de Hamming:

$$\sum_{i=0}^{t_c} \binom{n}{i} \leq 2^{n-k}$$

Reemplazando nuestros valores ($n = 14, k = 10, n - k = 4$):

- Si evaluamos para $t_c = 1$:

$$\binom{14}{0} + \binom{14}{1} = 1 + 14 = 15$$

$$15 \leq 2^4$$

$$15 \leq 16$$

Podemos observar que la cota se cumple.

- Si evaluamos para $t_c = 2$:

$$\binom{14}{0} + \binom{14}{1} + \binom{14}{2} = 1 + 14 + 91 = 106$$

$$106 \leq 2^4$$

$$106 \leq 16$$

Para este caso, el resultado supera el límite de 16.

Por lo tanto, el código diseñado puede corregir, como máximo, 1 error simple por bloque.

2.1.2. Diseño de la matriz de verificación de paridad (H^T)

Para alcanzar la capacidad de corrección de $t_c = 1$, el código requiere una distancia mínima $d_{min} \geq 3$. La d_{min} está dada por la cantidad mínima de filas de H^T que sumadas (en módulo 2) dan como resultado el vector nulo. Para garantizar que ninguna combinación de 1 o 2 filas dé cero, se deben cumplir dos condiciones:

- Ninguna fila puede ser el vector nulo.
- Todas las filas deben ser distintas.

Como el código es sistemático, la matriz se estructura como:

$$H^T = \begin{bmatrix} P_{10 \times 4} \\ I_{4 \times 4} \end{bmatrix}$$

Existen $2^4 - 1 = 15$ combinaciones no nulas de 4 bits. Descartando las 4 combinaciones correspondientes a la matriz identidad $I_{4 \times 4}$, se seleccionan 10 de las 11 combinaciones restantes para conformar la submatriz de paridad $P_{10 \times 4}$, resultando en la siguiente matriz de dimensiones 14 x 4:

$$H^T = \begin{bmatrix} 0 & 0 & 1 & 1 \\ 0 & 1 & 0 & 1 \\ 0 & 1 & 1 & 0 \\ 0 & 1 & 1 & 1 \\ 1 & 0 & 0 & 1 \\ 1 & 0 & 1 & 0 \\ 1 & 0 & 1 & 1 \\ 1 & 1 & 0 & 0 \\ 1 & 1 & 0 & 1 \\ 1 & 1 & 1 & 0 \\ \hline 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

2.1.3. Diseño de la matriz generadora (G)

A partir de la submatriz $P_{10 \times 4}$ definida en el paso anterior, se construye la matriz generadora del código sistemático concatenando la matriz identidad $I_{10 \times 10}$, obteniendo la estructura:

$$G_{10 \times 14} = [I_{10 \times 10} | P_{10 \times 4}]$$

La matriz resultante es:

$$G = \left[\begin{array}{cccccccccccc|cccc} 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 1 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 1 & 0 \end{array} \right]$$

2.1.4. Capacidad de detección de errores (t_d)

La cantidad máxima de errores que el código puede detectar con seguridad (t_d) está directamente vinculada a su distancia mínima mediante la siguiente relación:

$$t_d = d_{min} - 1$$

Dado que el diseño propuesto en la sección 2.1.2 garantiza una distancia mínima $d_{min} = 3$, la capacidad de detección resulta en:

$$t_d = 3 - 1 = 2$$

Esto implica que si el canal introduce exactamente 2 errores en una palabra código de 14 bits, el síndrome obtenido será distinto de cero, permitiendo detectar la presencia de errores durante la recepción. Sin embargo, como la capacidad de corrección del código es ($t_c = 1$), el decodificador no puede corregir dos errores simultáneos. En consecuencia, el síndrome generado por dicho patrón de error será interpretado como si correspondiera a un único error, provocando una corrección incorrecta y, por ende, una decodificación errónea de la palabra transmitida.

2.1.5. Ganancia asintótica de codificación (G_a)

La ganancia de codificación asintótica (G_a) establece el límite teórico de la mejora de desempeño del sistema codificado frente al sistema sin codificar. Matemáticamente, se define como el valor al que tiende la ganancia de código (G_c) cuando la probabilidad de error de bit tiende a cero ($P_{eb} \rightarrow 0$) ó, de forma

equivalente para un canal AWGN, cuando la relación señal a ruido tiende a infinito ($E_b/N_0 \rightarrow \infty$).

Considerando que el sistema utiliza detección dura, la ganancia asintótica teórica se determina a partir de la tasa del código (k/n) y su distancia mínima (d_{min}):

$$G_{a(dura)} = \frac{k}{n} \left(\frac{d_{min} + 1}{2} \right)$$

Reemplazando con los parámetros del código (14,10) diseñado ($k = 10, n = 14, d_{min} = 3$):

$$G_{a(dura)} = \frac{10}{14} \left(\frac{3 + 1}{2} \right) = \frac{10}{14}(2) = \frac{20}{14} \approx 1.428$$

Expresar este valor en decibeles (dB), servirá como referencia para el análisis y comparación de los resultados obtenidos mediante simulación:

$$G_{a(dura)dB} = 10 \log_{10}(1.428) \approx 1.55dB$$

2.2. Inciso ii: Simulación y relevamiento de curvas

Para obtener las curvas de desempeño, se implementó un simulador iterando sobre la transmisión de 100.000 palabras de código a través de un canal modelado con Ruido Blanco Gaussiano Aditivo (AWGN) y modulación BPSK.

Una consideración para el diseño de la simulación es la normalización de la energía. Para que la comparación entre el sistema codificado y el sistema sin codificar sea equitativa, ambos deben utilizar la misma energía de bit de fuente (E_{bf}). Dado que el sistema codificado transmite $n = 14$ bits físicos de canal por cada $k = 10$ bits de fuente original, la energía de cada símbolo transmitido (E_s) debe escalarse proporcionalmente para no inyectar potencia de transmisión extra al sistema. Según el modelo matemático establecido, esta relación es:

$$E_{bf} = E_s \left(\frac{n}{k} \right)$$

Adoptando una energía normalizada $E_{bf} = 1$ para simplificar el modelo, la amplitud de la señal BPSK (A) se configuró algorítmicamente como:

$$A = \sqrt{\frac{k}{n} E_{bf}}$$

$$A = \sqrt{\frac{k}{n} \cdot 1} \implies A = \sqrt{\frac{k}{n}}$$

De esta forma, el eje de abscisas de las curvas resultantes representa la relación señal a ruido de la información pura.

La señal modulada se transmitió a través del canal sumando la componente de ruido aleatorio. En la etapa de recepción se aplicó detección dura, donde la decisión sobre cada bit recibido se realizó evaluando el signo de la componente real de la señal: los valores positivos se asociaron al bit lógico 1 y los negativos al bit lógico 0. Posteriormente, se procedió al cálculo matricial del síndrome para efectuar la corrección de errores simples.

Se calcularon las curvas teóricas de probabilidad de error propias del código operando como corrector. Para la probabilidad de error de palabra (P_{ep}), se empleó la sumatoria que evalúa la probabilidad de que el canal introduzca más errores que la capacidad de corrección del código ($t_c = 1$), donde p es la probabilidad de error de un símbolo en el canal:

$$P_{ep} = \sum_{i=t_c+1}^n \binom{n}{i} p^i (1-p)^{n-i}$$

Por su parte, la probabilidad de error de bit de fuente (P_{ebf}) se calculó utilizando la expresión basada en el error de palabra:

$$P_{ebf} = \left(\frac{2t_c + 1}{n} \right) P_{ep}$$

Finalmente, se graficaron los resultados de la simulación junto a estas curvas. Esta comparativa de desempeño se presenta a continuación en la **Figura 1**.

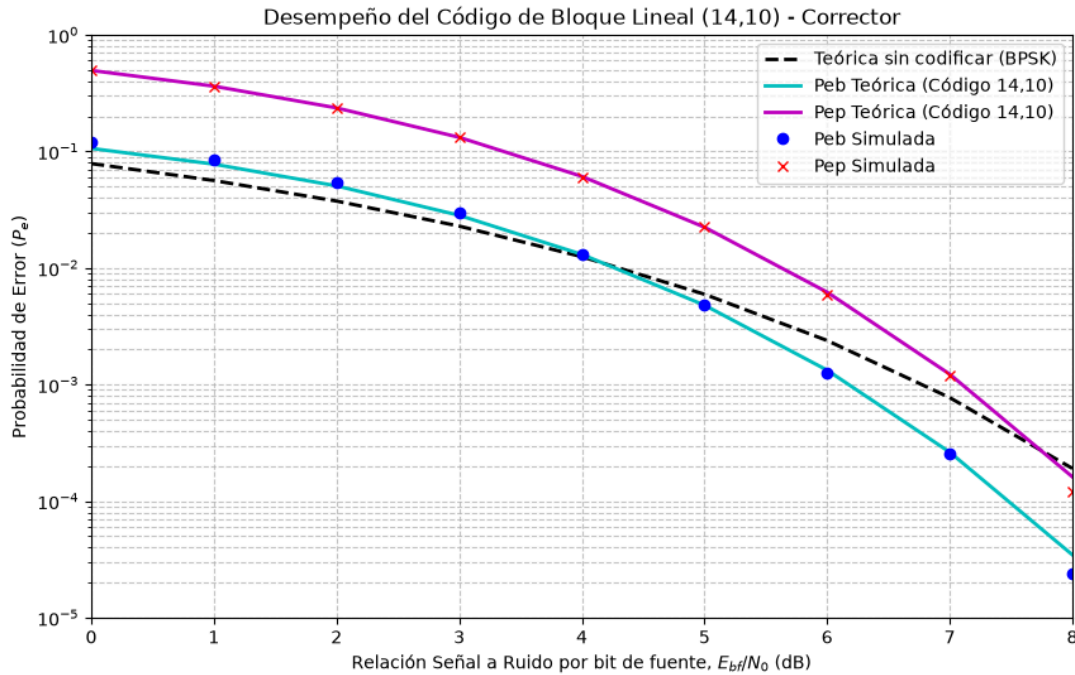


Figura 1. Curvas de probabilidad de error para el código operando como corrector.

2.3. Inciso iii: Análisis de desempeño

A partir de la visualización de las curvas presentadas en la **Figura 1**, el análisis del desempeño del sistema estructurado bajo la métrica de ganancia de código (G_c) revela tres regiones operativas fundamentales:

- Zona de ganancia negativa (penalidad por decisión dura):** Para relaciones señal a ruido inferiores a $E_b / N_0 \approx 4 \text{ dB}$, la curva del sistema codificado se ubica por encima de la teórica. En esta región, la ganancia de código es estrictamente negativa. Esto se debe a que la alta densidad de errores del canal supera la capacidad de corrección ($t_c = 1$), provocando que el decodificador realice alteraciones incorrectas e introduzca más errores de los que corrige.
- Zona de ganancia efectiva (operación práctica):** Superado el punto de cruce de 4 dB , el sistema comienza a evidenciar una ganancia de código positiva. Si se toma como referencia de diseño una probabilidad de error de bit de $P_{eb} = 10^{-3}$, el sistema BPSK original requiere invertir aproximadamente $6,7 \text{ dB}$ de potencia. El sistema codificado alcanza esa misma confiabilidad utilizando sólo $6,1 \text{ dB}$. Por ende, el código aporta una ganancia práctica de $G_c \approx 0,6 \text{ dB}$ para dicho requerimiento, permitiendo ahorrar energía de transmisión.

- **Zona de convergencia (límite asintótico):** Al evaluar el comportamiento de la ganancia en relaciones señal a ruido $E_b / N_0 > 6 \text{ dB}$, se observa que la separación horizontal entre la curva teórica y la curva simulada (P_{eb}) continúa expandiéndose. Esto respalda el análisis analítico previo, confirmando que, a medida que la probabilidad de error tiende a cero, la ganancia de código (G_c) se aproximará gradualmente a su límite de ganancia asintótica ($G_a \approx 1,55 \text{ dB}$).
- **Validación del modelo teórico:** Se observa que los puntos discretos obtenidos mediante la simulación (marcadores azules y rojos) se superponen con exactitud milimétrica sobre las curvas teóricas continuas calculadas analíticamente (líneas cian y magenta).

3. Ejercicio 2: Código de bloque lineal (14,10) como detector

3.1. Inciso i: Lógica de detección y parámetros operativos

El diseño analítico de este esquema se fundamenta en las mismas matrices generadora (G) y de verificación de paridad (H^T) propuestas y justificadas en las secciones 2.1.2 y 2.1.3. La estructura matemática del código (14,10) permanece inalterada; la modificación metodológica reside exclusivamente en el tratamiento del síndrome durante la etapa de recepción.

Al operar como detector de errores, se anula voluntariamente la capacidad de corrección del sistema ($t_c = 0$). De acuerdo con la relación teórica $t_d = d_{min} - 1$, el sistema ahora es capaz de detectar con total seguridad hasta $t_d = 2$ errores simultáneos dentro de una misma palabra de código.

A diferencia del esquema corrector, el algoritmo no intentará correlacionar el síndrome con las filas de H^T para buscar la posición del defecto e invertir el bit; la palabra será directamente marcada como errónea e irrecuperable.

Finalmente, cabe destacar que al prescindir de la corrección de bits, el concepto de Ganancia Asintótica (G_a) calculado en la sección 2.1.5 carece de aplicación en este escenario. El objetivo de este esquema de detección no es mejorar la tasa de error

de bit del canal (P_{eb}), sino garantizar una alta confiabilidad e integridad en los bloques de información que el decodificador da por válidos.

3.2. Inciso ii: Simulación y relevamiento de curvas

La evaluación del código como detector se realizó adaptando el simulador desarrollado para el **Ejercicio 1**. Se mantuvieron idénticos los parámetros fundamentales del canal AWGN, la modulación BPSK y la normalización de la energía.

La modificación se implementó en la lógica del receptor: tras obtener el síndrome matricial y clasificar las palabras como válidas o corruptas, el algoritmo omite la etapa de corrección. Los bits de fuente se extraen directamente de la palabra demodulada con decisión dura, conservando intactos los errores introducidos por el canal.

A diferencia del esquema corrector, el parámetro de interés principal es la probabilidad de error de palabra (P_{ep}). Esta curva se calcula utilizando la capacidad de detección ($t_d = 2$):

$$P_{ep} = \sum_{i=t_d+1}^n \binom{n}{i} p^i (1-p)^{n-i}$$

Esta comparativa de desempeño se presenta a continuación en la **Figura 2**.

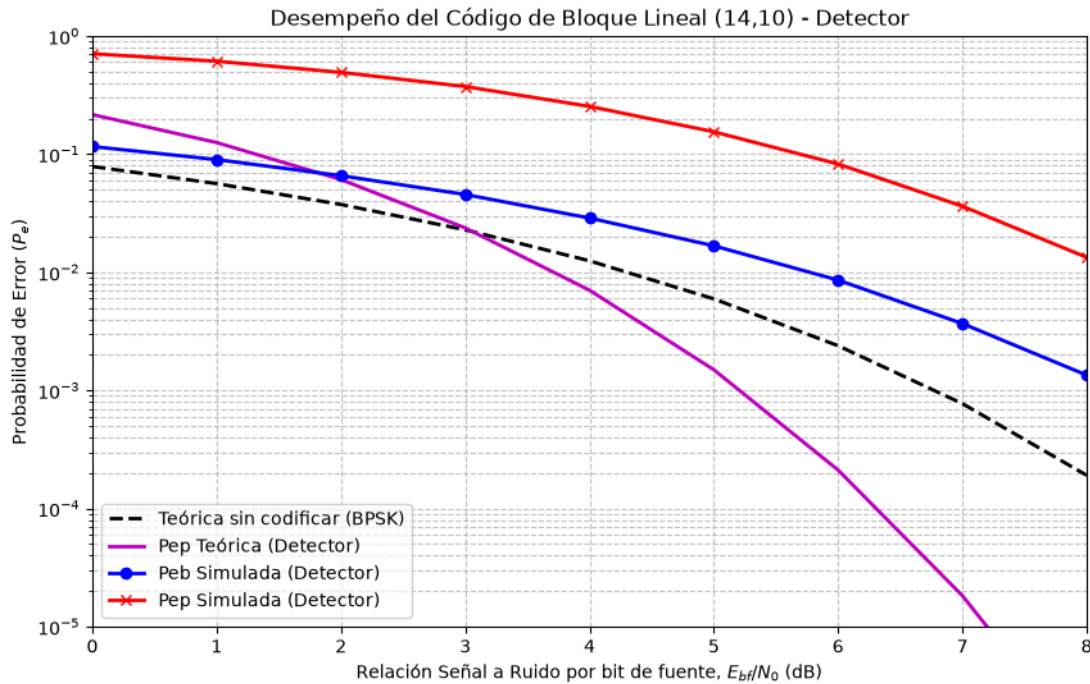


Figura 2. Curvas de probabilidad de error para el código operando como detector.

3.3. Inciso iii: Análisis de desempeño

El análisis visual de las curvas presentadas en la **Figura 2** expone un comportamiento distinto al evaluado en el esquema corrector, destacándose las siguientes observaciones:

- **Ausencia de ganancia de código:** La curva de probabilidad de error de bit (P_{eb}) del sistema codificado se ubica de manera estricta por encima de la curva teórica (BPSK) para todo el rango de E_b / N_0 simulado. Esto implica una ganancia de código (G_c) permanentemente negativa.
- **Penalidad energética pura:** La degradación observada en la tasa de error responde a que el sistema asume la penalidad energética de transmitir redundancia, pero sin aprovechar dicha redundancia para revertir los errores. En consecuencia, los símbolos físicos son más vulnerables al ruido AWGN y la tasa de error del canal empeora.
- **Integridad vs. Tasa de error:** Aunque el gráfico muestra una probabilidad de error superior al sistema original, la utilidad de un detector puro no se mide por la cantidad de bits que salva. El beneficio real radica en el uso del síndrome como barrera de control: al no forzar correcciones que podrían ser erróneas, el sistema detecta e invalida con certeza los paquetes corruptos.

- **Tasa de rechazo vs. Errores no detectados:** Se destaca una separación notoria entre la curva de probabilidad de error de palabra simulada (roja) y su contraparte teórica (magenta). La curva simulada contabiliza todos los bloques descartados por presentar al menos un error físico en el canal (≥ 1 error). Por el contrario, la curva teórica representa la probabilidad límite de ocurrencia de 3 o más errores simultáneos (superando $t_d = 2$), escenario donde el síndrome podría resultar nulo engañando al decodificador.

4. Ejercicio 3: Codificación de fuente mediante algoritmo de Huffman

4.1. Implementación del algoritmo y estimación de probabilidades

Para llevar a cabo la compresión del archivo “**logo FI.tif**”, se desarrolló un script que procesa la imagen asegurando un formato binario estricto (1 bit por píxel). Esto permite obtener una secuencia de información unidimensional conformada por un alfabeto base de dos símbolos discretos ($m_0 = 0$ para negro y $m_1 = 1$ para blanco).

Para estimar las probabilidades de los mensajes, la secuencia original se segmentó en bloques contiguos de tamaño n , correspondiendo a $n = 2$ y $n = 3$ para conformar las fuentes extendidas de orden 2 y orden 3 respectivamente. La probabilidad de cada mensaje extendido se calculó determinando su frecuencia relativa, es decir, el cociente entre la cantidad de apariciones de dicho bloque y el total de bloques conformados en la imagen.

Con los valores de las probabilidades a priori estimados, se implementó el algoritmo de Huffman utilizando una estructura de datos basada en colas de prioridad. El procedimiento consistió en construir iterativamente un árbol binario, agrupando y sumando en cada paso los dos nodos de menor probabilidad hasta conformar una única raíz. El recorrido descendente de este árbol permitió generar la regla de codificación unívoca, garantizando la asignación de palabras de código más cortas a los bloques de píxeles con mayor probabilidad de ocurrencia.

4.2. Inciso i: Longitud promedio y tasa de compresión

El largo promedio se determinó como el cociente entre el largo total de la secuencia codificada y la cantidad de bits de la fuente original. Por su parte, la tasa de

compresión se calculó como la relación inversa, es decir, el cociente entre la longitud de la secuencia original sin codificar y la longitud de la secuencia codificada.

Tras ejecutar el algoritmo de codificación sobre la imagen, se obtuvieron los siguientes resultados para ambas extensiones:

- Fuente extendida de orden 2:
 - Largo promedio: *0,7583 bits de código / bit de fuente.*
 - Tasa de compresión: *1,3187*
- Fuente extendida de orden 3:
 - Largo promedio: *0,5328 bits de código / bit de fuente.*
 - Tasa de compresión: *1,8770*

Estos resultados numéricos evidencian que, al incrementar el orden de extensión de la fuente (pasando de $n = 2$ a $n = 3$), se reduce la cantidad promedio de bits requeridos para transmitir cada bit de información original, logrando así un aumento significativo en la tasa de compresión del archivo.

4.3. Inciso ii: Análisis de compresión con fuente extendida

El análisis de la fuente original demostró que los píxeles blancos y negros presentan probabilidades de ocurrencia muy similares (aproximadamente 51,4% y 48,5% respectivamente). Si la imagen fuese una fuente de información sin memoria, es decir, si cada píxel fuese totalmente independiente de su entorno, la probabilidad de los bloques extendidos se distribuiría de manera uniforme y la compresión sería prácticamente nula.

Sin embargo, el éxito de la compresión radica en que los píxeles de una imagen gráfica poseen una fuerte correlación espacial. La ocurrencia de un color condiciona al entorno: si un píxel es blanco (parte del fondo continuo de la imagen), es altamente probable que el píxel adyacente también lo sea. Lo mismo ocurre con los píxeles negros que conforman el cuerpo del logo de la facultad.

Al aplicar la extensión de la fuente (por ejemplo, a orden 2), el algoritmo agrupa estos píxeles contiguos y logra capturar dicha dependencia espacial. Esto se refleja directamente en las probabilidades estimadas en el inciso anterior: los bloques de colores continuos como '11' (dos blancos) y '00' (dos negros) concentran casi la totalidad de las apariciones, sumando más del 97% de la probabilidad total. En

contraparte, las transiciones de color espacial ('01' y '10') resultan ser eventos poco frecuentes.

En consecuencia, el algoritmo de Huffman explota eficientemente esta distribución desigual. Al asignar palabras de código de longitud mínima (1 o 2 bits) a los bloques altamente probables, y secuencias más largas a las transiciones raras, se garantiza que la gran mayoría del mensaje sea codificado con muy pocos bits. Este mecanismo es el que permite reducir el largo promedio general y lograr una compresión efectiva a pesar de la equiprobabilidad estricta de los símbolos base individuales.

5. Conclusiones

Mediante la codificación de canal, se comprobó cómo la introducción sistemática y controlada de redundancia permite a un sistema sobreponerse a las perturbaciones del medio físico (ruido AWGN). Se evidenció el compromiso de diseño de estos esquemas: el uso de códigos correctores mejora la tasa de error a costa de una penalidad energética inicial, mientras que los códigos detectores priorizan la integridad absoluta de la información rechazada frente a su corrección.

Por otro lado, la codificación de fuente demostró el principio opuesto: la compresión mediante la eliminación de la redundancia natural de los datos. La implementación del algoritmo de Huffman comprobó que la correlación espacial en las imágenes gráficas otorga un margen sustancial para la reducción del tamaño de los mensajes, logrando optimizar el uso de los recursos del canal. En conjunto, las simulaciones reafirman que el diseño óptimo de un sistema de telecomunicaciones requiere un balance entre comprimir la información para maximizar la eficiencia y protegerla para garantizar su confiabilidad.

Apéndice A. Implementación y equivalencia de lenguajes

Si bien la cátedra proporcionó expresiones de referencia en sintaxis de MATLAB, el simulador de este trabajo fue desarrollado íntegramente en Python utilizando las librerías **numpy** para el álgebra matricial y **scipy** para el cálculo de cotas teóricas. A continuación se detallan las equivalencias y optimizaciones implementadas respecto al modelo base:

- **Simulación del canal y ruido AWGN:** El modelo de MATLAB propone generar ruido gaussiano complejo: ***noise = sqrt(N0/2)*(randn(1,n) + 1i*randn(1,n));***
Dado que el sistema utiliza modulación BPSK y detección dura evaluando únicamente el signo de la componente real (***real(r) > 0***), en Python se optimizó el costo computacional simulando exclusivamente la dimensión real del ruido: ***noise = np.sqrt(N0 / 2) * np.random.normal(0, 1, s.shape)***
- **Detección dura e indexación matricial:** La conversión lógica a valores enteros sugerida en MATLAB (***vr = 1*(real(r) > 0)***) se replicó en Python utilizando el casteo nativo de la librería ***numpy***: ***vr = (r > 0).astype(int)***. Asimismo, las operaciones de aritmética de módulo 2 se resolvieron mediante el operador nativo ***% 2***.
- **Búsqueda de síndromes:** En lugar de utilizar la función ***bi2de*** sugerida para convertir bloques binarios a decimales y compararlos, la corrección de errores en Python se implementó mediante máscaras booleanas (***np.where(np.all(H_T == syndrome_actual, axis=1))***). Esto permite encontrar la coincidencia exacta de las filas en la matriz traspuesta ***H^T*** operando puramente a nivel matricial.
- **Procesamiento de la fuente (Ejercicio 3):** Finalmente, para la codificación de fuente, se prescindió de la función ***imread*** de MATLAB y se utilizó el módulo ***Image*** de la librería ***Pillow*** (PIL) en Python. Al invocar el método ***.convert('1')***, se garantizó que la matriz de la imagen gráfica fuese tratada estrictamente como un arreglo lógico de 1 bit por píxel (blanco y negro puro), cumpliendo con los requisitos de la fuente discreta binaria analizada.

Apéndice B. Guía de ejecución de los simuladores

Para facilitar la revisión y reproducción de los resultados obtenidos en este trabajo práctico, se detalla a continuación la guía de configuración y ejecución de los scripts desarrollados en lenguaje Python. Se asume que el evaluador cuenta con una instalación previa de Python (versión 3.6 o superior) y el gestor de paquetes **pip** configurado en su variable de entorno.

B.1. Instalación de dependencias

Los simuladores hacen uso de librerías de cálculo científico, ploteo y procesamiento de imágenes que no vienen por defecto en la instalación estándar de Python. Para instalarlas, abra una terminal (Símbolo del sistema, PowerShell o Bash) y ejecute el siguiente comando:

```
pip install numpy scipy matplotlib pillow
```

B.2. Estructura de directorio

Para el correcto funcionamiento de los scripts, especialmente el del Ejercicio 3, es necesario que todos los archivos se encuentren en la misma carpeta. La estructura del directorio debe ser la siguiente:

- ***simulacion_ej1.py*** (Simulador del código corrector).
- ***simulacion_ej2.py*** (Simulador del código detector).
- ***simulacion_ej3.py*** (Simulador de compresión Huffman).
- ***logo FI.tif*** (Imagen original provista por la cátedra).

B.3. Ejecución y resultados esperados

Para iniciar las simulaciones, navegue desde la terminal hasta el directorio donde extrajo los archivos y ejecute los scripts individualmente:

- **Ejercicio 1:** Ejecutar python ***simulacion_ej1.py***. La terminal imprimirá el progreso de la simulación decibel por decibel. Al finalizar, se abrirá automáticamente una ventana emergente con la gráfica comparativa (**Figura 1**). El programa finalizará al cerrar dicha ventana.
- **Ejercicio 2:** Ejecutar python ***simulacion_ej2.py***. El comportamiento es idéntico al caso anterior, arrojando por pantalla las tasas de error y abriendo la gráfica correspondiente al esquema detector (**Figura 2**).

- **Ejercicio 3:** Ejecutar python ***simulacion_ej3.py***. Este script no genera gráficas, sino que imprime directamente en la consola el desglose analítico del archivo: el total de píxeles, las probabilidades, y para cada orden de fuente extendida (2 y 3), desplegará el largo promedio, la tasa de compresión final y la tabla completa del diccionario de Huffman generado.